

Obligatory assignment2 In3030 v25 alexsho

Introduction:

This report is showing the difference in speedup between sequential, transposed and parallel matrix multiplication. How run times differ depending on parallelization and cache locality.

My data is from running the program on Macbook air M2 (2022) using aj apple M2 chip. The chip has and 8-core CPU with 4 performance cores @3.49GHz and 4 efficiency cores @2.42GHz.

Sequential Matrix Multiplication:

The sequential matrix multiplication is made up of 3 loops. The first loop iterates over the rows in the first matrix, the second loop iterates over columns in the second matrix and the last loop sums the multiplication between the elements.

Parallel Matrix Multiplication:

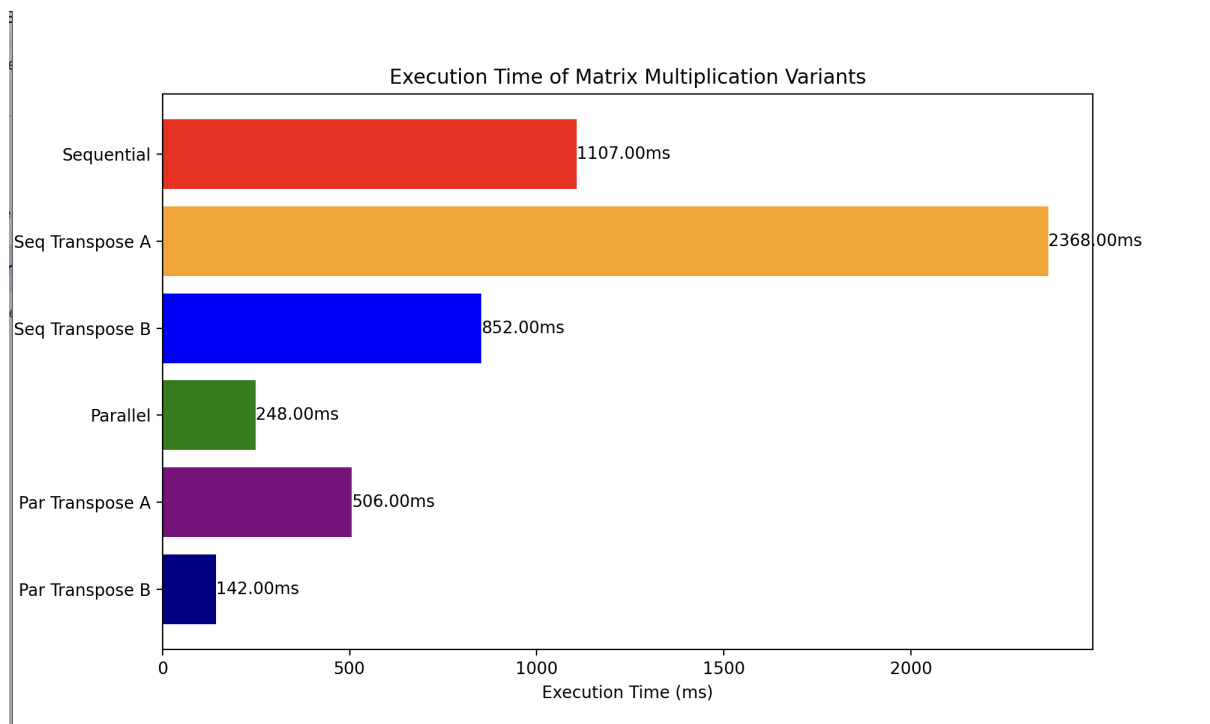
I implemented simple parallelization by distributing the computation in multiple threads. I divided the rows of the result matrix into partitions of the threads so that each thread had their part to compute. Due to the threads not sharing any variables and writes to their location in the result matrix any synchronization was not needed. However threads are terminated properly by awaiting before termination making sure that all the threads are finished with their task before termination.

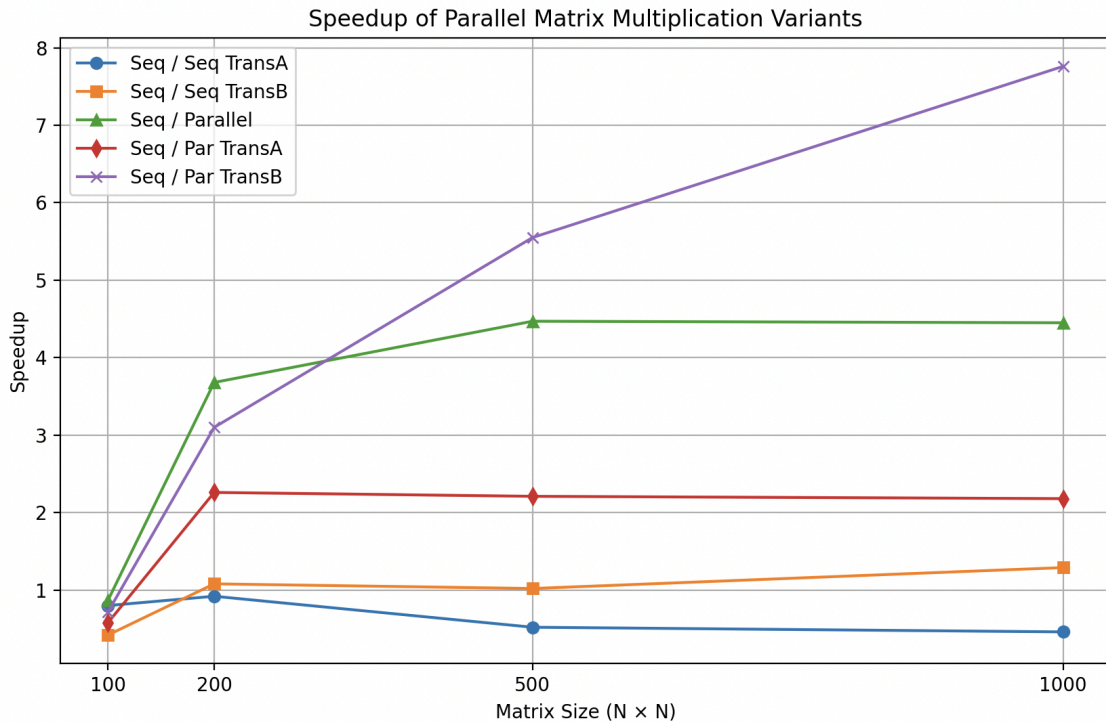
Measurements:

As seen in the graph and table given below we can see a big difference between the transpose of b and the transpose of a in both the parallel and sequential variants. The sequential variant with a transposed does not achieve speedup > 1 which means that the cache locality is worse than the untransposed version. On the other hand when transposing b the speedups are higher than one (except for $n = 100$ due to the transposing taking time) showing better cache locality.

When looking at the parallel variants we can see a speedup over 1 for all variants for $n = 200$, $n = 500$ and $n = 1000$. The reason why the transposed parallel variants did not get the speedup greater than one before $n = 200$ is because of the excessive overhead in the parallelization and the transposing of the matrixes. This means that even though overhead occurs when parallelization, hence why speedup < 1 for $n = 100$, the time saved with parallel computing and transposing outweigh the overhead somewhere between $n = 100$ and $n = 200$.

speedups against Sequential no transpose:					
NxN	100x100	200x200	500x500	1000x1000	
Speedup Sequential Transpose A	0,8	0,92	0,52	0,46	
Speedup Sequential Transpose B	0,42	1,08	1,02	1,29	
Speedup Parallel no Transpose	0,87	3,68	4,47	4,45	
Speedup Parallel Transpose A	0,58	2,26	2,21	2,18	
Speedup Parallel Transpose B	0,72	3,1	5,55	7,76	





User Guide:

Compile: `javac *.java`

Run: `java Main <seed> [size of matrix]`

- if size of matrix is not specified sizes [100x100, 200x200, 500x500, 1000x1000] are used.
- size of matrix has to be specified to validate for correct results

Example run 1: `java Main 42`

- runs for all predefined sizes, does not validate

Example run 2: `java Main 42 1000`

- runs for 1000x1000 and prints to terminal if results are correct

Conclusion:

In conclusion Transposing A makes the cache locality worse, making the runtime longer than the untransposed one and the Transposing of B makes the cache locality better leading to shorter runtimes. This is for both the sequential and the parallel variants of matrix multiplication.

Appendix:

All Java source files are included in the submitted zip (in3030_assignment2_alexsho.zip)

Seed used for gathering data for table and graph was retrieved from the run: java Main 42